

G.A.I.A
GESTURAL AND AUDITORY INTELLIGENT
ASSISTANT

Project report submitted in the partial fulfilment of
the requirements for the award of the degree of
MASTER OF COMPUTER APPLICATIONS

In
DEPARTMENT OF INFORMATION TECHNOLOGY AND COMPUTER
APPLICATIONS

A project report submitted by
DANTHINA UDAYSAI
(Regd. No. 323207360066)

Under the esteemed guidance of
Dr. G. SANDHYA DEVI ,Ph.D
Professor(Ad-Hoc), Department of CS&SE



DEPARTMENT OF INFORMATION TECHNOLOGY AND COMPUTER
APPLICATIONS

ANDHRA UNIVERSITY COLLEGE OF ENGINEERING (A)

ANDHRA UNIVERSITY

VISAKHAPATNAM – 530003 (2023 – 2025)

**DEPARTMENT OF INFORMATION TECHNOLOGY AND COMPUTER
APPLICATIONS**

ANDHRA UNIVERSITY COLLEGE OF ENGINEERING (A)

ANDHRA UNIVERSITY

VISAKHAPATNAM – 530003

CERTIFICATE



This is to certify that the project report entitled “G.A.I.A - GESTURAL AND AUDITORY INTELLIGENT ASSISTANT**” is a bonafide work carried out by **DANTHINA UDAYSAI** with Regd. No.: **323207360066**, a student of M.C.A. in AU COLLEGE OF ENGINEERING(A), ANDHRA UNIVERSITY, VISAKHAPATNAM, during the year 2023 – 2025, in partial fulfilment of the requirements for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**.**

DR. G. SANDHYA DEVI

Project Guide

Prof. KUNJAM NAGESWARA RAO

head of the department

TABLE OF CONTENTS

- ABSTRACT
- COMPUTATIONAL APPROACH
- EXISTING SYSTEM
- PROBLEM STATEMENT
- PROPOSED SYSTEM
- LITERATURE SURVEY
- SYSTEM ANALYSIS AND DESIGN
- SOFTWARE REQUIREMENT SPECIFICATION DOCUMENT
- DESIGN APPROACHES
- UML DESIGN OVERVIEW
- METHODOLOGY AND ALGORITHM
- IMPLEMENTATION
- RESULTS AND ANALYSIS
- CONCLUSION

ABSTRACT

In our digital world, intelligent personal assistants are changing the way we interact with technology, making our daily tasks easier and more intuitive. This project showcases an advanced AI-powered Voice Assistant system that combines voice commands, gesture recognition, natural language processing, and emotional responses, resulting in a truly interactive and dynamic digital companion. Built with Python and a variety of open-source libraries, this assistant is designed not just to handle everyday tasks but also to deliver an emotionally intelligent user experience through unique features like Roast Mode, Therapist Mode, and AI sarcasm responses.

The assistant listens to natural voice commands, recognizes wake words, and can perform a wide range of tasks such as opening apps, sending WhatsApp messages, playing music on YouTube, reading PDFs, taking screenshots, providing weather updates, translating text into multiple Indian and international languages, and searching Google or Wikipedia—all in a conversational manner. By integrating Google's Speech Recognition API, Pyttsx3, and Google Translate, it offers a powerful natural language interface.

One of the standout features of this assistant is Gesture Mode, which utilizes the MediaPipe and OpenCV libraries to interpret real-time hand gestures from a webcam feed. With gestures like a closed fist for play/pause, two fingers to open YouTube, or a pinky and index finger to launch WhatsApp, the assistant makes touchless control a reality. This feature not only enhances accessibility but also modernizes interaction, especially in situations where using voice commands might not be practical.

Additionally, the project introduces emotional intelligence modes, specifically Roast Mode and Therapist Mode. In Roast Mode, the assistant delivers playful, humorous jabs using a wide array of randomized, AI-inspired roasts that mimic genuine human sarcasm. On the flip side, Therapist Mode provides soothing and supportive responses, creating a pseudo-therapeutic experience for users.

Introduction:

This project introduces a smart, hands-free desktop assistant called JARVIS, built using Python.

It uses voice commands and hand gestures to interact with your PC in a natural way.

You can ask it to open apps, play music, translate languages, or check the weather.

The assistant responds using text-to-speech with gender-specific voices for local languages.

Using a webcam, it detects hand gestures to control music or switch modes.

It supports Therapist Mode to give calming, emotional support when needed.

JARVIS can switch between voice and gesture modes anytime using simple commands.

It handles tasks like adjusting volume, brightness, taking screenshots, and more.

Everything runs offline — no need for internet or APIs.

The assistant was built using tools like speech_recognition, pyttsx3, OpenCV, and MediaPipe.

Its modular design makes it easy to update, expand, and personalize.

Overall, it's designed to feel more like a friendly digital partner than just a robot.

Computational Approach

This project is all about blending Artificial Intelligence (AI), Machine Learning, and Computer Vision to develop a unique virtual assistant that you can control with your voice and gestures. It's designed to not just hear you, but to really understand and respond to what you say, your hand movements, and even the emotions behind your words. Plus, it can throw in some playful roasting or offer a bit of therapy-like support when needed. The assistant aims to replicate human-like conversation and behavior through a series of smart computational processes:

Voice Recognition (NLP + Speech Processing)

- **Library Used:** SpeechRecognition, Pyttsx3, Google Speech API
- The system captures audio input via a microphone and processes it using **Automatic Speech Recognition (ASR)**.
- It converts the speech signal into a text transcript using Google's speech-to-text services.
- Wake word detection ("Hello Jack", "Hey Mike") uses **keyword spotting** to initialize the assistant.

Processing Pipeline:

User Speech → Noise Filtering → STT Engine (Google API) → Command Parser
--

Natural Language Understanding (Command Parsing)

- Every voice command is linked to specific intent rules through straightforward keyword-based rule-matching algorithms.
- Complex conditions are assessed through if-elif blocks, allowing for a more natural flow in the way we phrase things.
- Advanced logic includes sentiment-sensitive functions like therapist mode, roast mode, and sarcasm mode.
- Commands are dynamically routed to system control, app automation, or web queries.

Gesture Recognition (Computer Vision + Mediapipe)

- Library Used: OpenCV, Mediapipe, PyAutoGUI
- The system captures video frames using the webcam and applies Mediapipe's hand landmark tracking model.
- It computes the relative positions of key hand landmarks (e.g., index finger tip, wrist, thumb) to classify gestures like:
 - Fist (play/pause)
 - Swipe left/right (next/previous song)
 - Two fingers (open YouTube)
 - Index + pinky (open WhatsApp)
- A gesture like three fingers raised switches the mode back to voice.

Processing Pipeline:

Camera Feed → Hand Detection (Mediapipe) → Landmark Coordinates → Gesture Rules → Action Mapping

Text-to-Speech Engine (TTS)

- Library Used: pyttsx3 for offline TTS
- The assistant responds using dynamic voice synthesis.
- Based on the language or command, it switches between male and female voices, especially during translations.
- The system supports multilingual feedback and emotional tone (e.g., gentle tone in therapist mode, harsh tone in roast mode).

Multilingual Translation System:

- Library Used: googletrans or deep-trans
- The assistant uses machine translation to translate user text into multiple Indian and foreign languages.
- Supports translation into Hindi, Telugu, Tamil, French, Spanish, Japanese, etc.
- It selects appropriate voice tones for each language and plays back the translated sentence.

Fun & Emotional Intelligence Modules:

- Therapist Mode: Uses a predefined set of motivational, emotional support phrases, chosen random or conditional triggers.
- This mode simulate emotional response patterns and make interactions more human-like.

Application & System Control (Automation):

- Library Used: os, pyautogui, psutil, subprocess
- The assistant can open and close apps (e.g., Notepad, Chrome, Word).
- Controls system settings like volume, brightness, shutdown, restart.
- Executes WhatsApp messaging, web searches, YouTube playback, and voice calls using automated browser interactions and GUI automation.

Wake Word Activation & Passive Listening:

- Passive listening logic listens continuously and waits for a wake phrase.
- After activation, it switches to command recognition mode for 30 seconds, else resets.
- This improves efficiency and reduces unwanted activation.

Custom Document Generation (Gesture Help Docs):

- Generates a dynamic gesture guide as a text file and opens it in Notepad upon entering gesture mode.
- It avoids camera frame overlay and provides a simple user-accessible reference.

Mode Management (Voice ↔ Gesture Switching):

- A global mode variable decides if the assistant is operating in gesture mode or voice mode.
- Each mode has separate control logic and feedback mechanisms.
- Certain gestures can switch the mode to voice, immediately halting gesture detection.

Existing System

In today's digital world, popular voice assistants like Siri, Alexa, Google Assistant, and Cortana help users perform tasks using voice commands. However, these systems have notable limitations. They lack emotional intelligence, often responding in a robotic, impersonal tone without understanding user feelings. There is no gesture support, making them less accessible to users with hearing or speech impairments. They also don't offer fun modes like roasting, therapy, or sarcasm that could make interactions more personal or entertaining. Customization is limited, with users unable to add their own commands or personalize responses. Additionally, these assistants rely heavily on internet connectivity, making them nearly unusable offline. Lastly, they don't support smooth switching between different interaction modes like voice and gestures, limiting flexibility and user experience.

Problem Statement

In the rapidly evolving world of artificial intelligence and human-computer interaction, traditional voice assistants like Siri, Alexa, and Google Assistant have made remarkable progress—but they still fall short in several crucial areas. These assistants are often limited to basic voice commands, lack emotional depth, and fail to adapt to diverse user needs, especially when it comes to accessibility, personality, and fun engagement.

Despite advancements in voice processing and automation, most existing systems operate in a rigid, task-oriented manner. They provide predefined responses, offer no custom modes like humor or therapy, and are unable to understand user emotions or intent beyond literal commands. Furthermore, they are almost exclusively dependent on voice input, leaving out users who might benefit from gesture-based controls—such as people with disabilities or those in hands-busy environments.

Additionally, traditional assistants are not capable of switching between different personalities or interaction styles—they can't roast you for fun, talk like a therapist when you're down, or shift to gesture mode when voice input isn't possible. There's no personalization, no humor, no empathy—and most importantly, no human-like interaction that feels real, flexible, or fun.

- Voice assistants are too robotic, with no emotional depth.
- They don't support alternate inputs like hand gestures.
- They lack modes like roasting, sarcasm, or emotional support (therapy).
- There's no regional voice personalization (like female voice for Indian languages).
- Interaction feels like commanding a machine—not like talking to a friend.

Proposed System

The proposed system is a next generation AI assistant that breaks free from the limitations by combining voice control, gesture recognition, and emotional intelligence. This voice assistant isn't just about executing tasks. Whether you're looking for productivity, emotional support, or just a good laugh, the assistant adapts to your mood and context using smart modes like:

- **Therapist Mode** – Offers calm, supportive responses like a caring listener.
- **Gesture Mode** – Control music and open apps with your hand movements using camera-based recognition.
- **Language Translator** – Translates any word or phrase into Indian or foreign languages using correct TTS and appropriate male/female voice.
- **YouTube Integration** – Play any song instantly with a voice command.
- **WhatsApp Controls** – Open WhatsApp, send messages, and even initiate calls.
- **Screenshot, File Search, Notes, Weather, Google Search, System Control, and more** – All done hands-free with natural language.

Key Features:

1. Wake Word Activation:

Just say *"Hello Jack"* or *"Hey Mike"* to activate the assistant—no button-pressing or manual input.

2. Voice-Only and Gesture-Only Modes:

You can switch between voice and gesture mode anytime. It supports accessibility for users with

speaking or hearing challenges.

3. Emotion-Based Personality Switching:

Unlike boring assistants, this one can roast you for fun, switching to comfort therapist replies, or get sarcastic depending on what you say.

4. Custom TTS (Text-to-Speech):

Female voices for Indian languages and male for foreign ones give it a realistic feeling.

5. Crazy Fun Features:

Like reverse command guessing, AI insult battles, and roast/therapy switching make the interaction enjoyable for students, developers.

6. Smart Integration:

The assistant can open and close apps like Notepad, Chrome, Word, WhatsApp, etc., control volume/brightness, and even shut down or restart the system—all by voice.

7. On-Screen Outputs:

Important responses (weather, translations, jokes, etc.) are not just spoken but also printed in the console for clarity and logs.

Literature Survey:

[JARVIS: An interpretation of AIML with integration of gTTS and Python Ravivanshikumar Sangpal, Tanvee Gawand, Sahil Vaykar, and Neha Madhavi, of Computer Technology, Government Polytechnic Pen, 402 107, Maharashtra, India, 2019]

The integration of Artificial Intelligence (AI) and Natural Language Processing (NLP) has revolutionized human-machine interaction, giving rise to popular voice assistants like Google Assistant, Alexa, Siri, and Cortana. These tools are great for executing tasks such as setting alarms or playing music but are limited in emotional depth, gesture control, and personalization. While platforms like Google Assistant leverage powerful AI frameworks, they lack support for fun or emotion-aware interaction. Meanwhile, individual developers have attempted to build Jarvis-like assistants using Python and basic libraries like `speech_recognition`, `pyttsx3`, and `pyautogui`, but these projects typically focus only on voice automation

and lack multi-dimensional features like gesture recognition or mood-specific modes (e.g., roast or therapist mode). Research on multimodal AI at institutions like MIT and Stanford has explored combining voice, gestures, and facial expressions to enhance interactions, yet these approaches are often costly and hardware-dependent. Libraries like MediaPipe and OpenCV have enabled real-time gesture control, but most use cases remain separate from voice assistants. Overall, existing systems either prioritize task execution or require advanced setups, with no unified platform offering voice, gesture, and emotional intelligence—alongside fun interaction modes and offline compatibility—all powered by simple, accessible technologies.

System Analysis and Design:

System Analysis (Understanding the Needs)

Before building our voice assistant, we took a step back to understand what users actually need and expect from a modern assistant. Traditional assistants (like Alexa or Siri) are great at doing tasks like setting alarms or giving weather updates—but they often feel **robotic**, **impersonal**, and **boring**.

Many users—especially students and everyday PC users—want something more:

- A system that listens and responds with personality (like roasting or comforting).
- An assistant that doesn't just talk back—but also **acts**, by opening apps, playing music, sending WhatsApp messages, and more.
- A mode where gestures can be used without even speaking, useful in noisy environments or for users who are physically challenged.
- A sense of fun, sarcasm, or empathy, based on mood and intent.

That's where our system comes in.

Design Approach

The project follows a modular design where each feature—like voice control, gesture recognition, roast mode, or therapist responses—is developed as an individual component, making the system easy to build, update, and scale. The assistant starts by listening to user commands through a microphone using the `speech_recognition` library, while MediaPipe and OpenCV are used to capture and interpret hand gestures like a fist or finger patterns for actions such as play/pause music or switching modes. Once input is detected, it is sent to a central logic layer that acts as the brain of the assistant—deciding what the user wants, whether that's launching YouTube, telling a joke,

or activating roast mode. If the input is confusing, the assistant responds with humor or a fallback message instead of just saying “I don’t understand,” making the interaction more natural. The output is handled through both voice and visual feedback. With `pyttsx3`, the assistant speaks back to the user and even changes to a female voice for more expressive or language-specific responses like translations in Indian languages. Visually, it can open Notepad or Word to display gesture guides, motivational quotes, or notes dictated by the user. Behind the scenes, the assistant also handles all the physical tasks like opening and closing apps, controlling brightness and volume, sending pre-formatted WhatsApp messages, taking screenshots, and performing quick Google searches. All these components work in sync, creating a smooth experience where the user feels like they are interacting with a real companion rather than a basic AI bot. The system is built to be scalable, lightweight, and fun—perfect for both everyday use and as a tech demo that surprises and entertains.

Design Philosophy:

We followed these key principles during design:

Principle	Meaning
User-friendly	Simple commands, easy to remember
Multi-modal	Works with both voice and gestures
Personality-rich	Offers humor, therapy, and mood-based replies
Modular	Each feature is separated, so we can add/remove easily
Offline Capable	Doesn't need constant internet connection (except for web-based actions)

Mode Switching Logic:

We built **two main modes**:

1. **Voice Mode** (default): Responds to spoken commands.

2. **Gesture Mode:** When gesture mode is enabled, the webcam opens, and the assistant watches for hand signs.

It can **switch between these modes** using voice commands like:

- "Switch to gesture mode"
- "Back to voice mode"

Software Requirements Specification (SRS)

Project Title: JARVIS: A Voice and Gesture-Controlled AI Assistant with Emotional Intelligence

Prepared By: ravivanshikumar sangpal, tanvee gawand, sahil yavkar and neha madhavi

Department: Computer Science

Year: 2019

The JARVIS voice-controlled AI assistant is a smart desktop application designed to automate everyday computer tasks through natural voice and gesture-based commands. Developed using Python, it enables users to control their PC hands-free, whether it's launching applications, managing files, controlling music, or even switching system modes—all just by talking or waving a hand. What sets JARVIS apart is its blend of productivity with personality: it comes equipped with mood-based interaction modes like Roast, Sarcasm, and Therapist, each offering a unique tone of communication. Whether you're in the mood for a joke, a motivational line, or a good-natured roast, JARVIS adapts to you. The assistant activates when it hears a wake word like "Hello Jack" and can smoothly switch between voice and gesture modes depending on the user's preference. Using a webcam, it detects hand gestures to perform tasks like skipping music tracks or opening WhatsApp. JARVIS is entirely offline and self-contained—built for Windows machines without relying on external cloud APIs, making it accessible in all conditions. It also includes support for Indian and foreign language translation, with voice tones switching automatically to suit the culture—female for Indian, male for foreign. Users don't need to be tech experts; even students or accessibility users with basic computer knowledge can operate the assistant with ease. From multimedia control and Google search to WhatsApp messaging, real-time language translation, system commands, and more, JARVIS aims to make human-computer interaction more intuitive, expressive, and fun than ever before.

1. Functional Requirements

<u>ID</u>	<u>Function</u>	<u>Description</u>
FR1	Voice Command Recognition	Recognize voice commands and execute corresponding actions
FR2	Gesture Recognition	Detect specific hand gestures and perform mapped functions
FR3	Roast Mode	On activation, responds with funny insults
FR4	Therapist Mode	On activation, responds with comforting statements
FR5	App Automation	Opens/closes apps like Notepad, Chrome, WhatsApp
FR6	Multimedia Control	Play, pause, next, previous music via voice/gesture
FR7	Language Translation	Translate spoken text to Indian/foreign languages
FR8	Wake Word Detection	“Hello Jack” or “Hey Mike” activates the assistant
FR9	Google Search	Voice-based web search using Chrome
FR10	WhatsApp Messaging	Send messages or make calls using voice

FR11	Document Display	Show gesture instructions as a document when gesture mode is activated
------	------------------	--

2. Non-Functional Requirements

<u>ID</u>	<u>Requirement</u>	<u>Description</u>
NFR1	Usability	Simple and intuitive voice-based interaction
NFR2	Accessibility	Gesture control for physically challenged users
NFR3	Performance	Voice command response within 2 seconds
NFR4	Reliability	Functions offline with robust error handling
NFR5	Compatibility	Runs on Python 3.8+ and Windows 10/11
NFR6	Maintainability	Code is modular for easy feature addition

The JARVIS voice-controlled AI assistant is a smart, offline, Python-based desktop application that allows users to interact with their computer through natural voice commands and hand gestures. It is designed to simplify everyday tasks such as opening or closing applications, controlling system settings like volume and brightness, taking screenshots, searching Google, translating languages, and sending WhatsApp messages—all without touching the keyboard or mouse. What makes JARVIS unique is its integration of personality-driven features like Roast Mode, Sarcasm Mode, and Therapist Mode, which provide dynamic, entertaining, or emotionally supportive responses based on the user's needs. The assistant listens for a specific wake word—like “Hello Jack”—to activate, ensuring it’s always ready but never intrusive. Gesture Mode, enabled through a webcam and powered by MediaPipe and OpenCV, allows users to control music playback or launch applications using intuitive hand signs. Unlike cloud-dependent systems, JARVIS runs completely offline, ensuring privacy and uninterrupted service even without an internet connection. It also supports language translation with automatic voice tone

switching—responding in a female voice for Indian languages and a male voice for foreign languages, enhancing its cultural adaptability. Designed with accessibility and inclusivity in mind, JARVIS is perfect for students, developers, individuals with disabilities, or anyone who prefers hands-free interaction. It provides a seamless transition between voice and gesture controls, giving users a multi-modal experience. With its balance of productivity, fun, and emotional intelligence, JARVIS turns your desktop into a truly interactive personal companion—making tech feel more human, engaging, and powerful

Design Approaches

Designing an intelligent voice and gesture-based assistant like JARVIS involves a carefully modular architecture where every functionality—from taking screenshots to translating languages—is implemented as an independent function, allowing for easy debugging, maintenance, and feature expansion. The core of the system lies in voice command processing, which starts with wake word detection like “Hello Jack,” followed by speech recognition via the `speech_recognition` library to convert audio into text, and then matching that text to predefined commands. Once matched, an execution engine calls the appropriate function, supporting natural language flexibility and providing spoken feedback via `pyttsx3`. Complementing voice control, the assistant incorporates gesture recognition using a webcam and libraries like MediaPipe and OpenCV, enabling actions like music control or app launching through real-time hand landmark detection without interrupting other system tasks. A key highlight is the mode-based design, allowing users to switch between Voice Mode, Gesture Mode, Therapist Mode, and Sarcasm Mode, with the assistant remembering and adjusting its behavior based on the active mode using global variables. Additionally, a contextual response system ensures dynamic personality adjustments, such as Roast Mode or supportive dialogue in Therapist Mode. For multilingual interaction, a translation engine powered by `googletrans` extracts and translates input while speaking the result using gender-specific voices—female for Indian languages and male for foreign ones. System automation tasks like opening apps, adjusting volume/brightness, and shutting down are smoothly executed through `subprocess` and `pyautogui`, with error handling in place to maintain smooth functionality. Underlying it all is a human-centered design philosophy, ensuring the assistant goes beyond mechanical responses to act as a supportive and entertaining digital companion, blending AI capabilities with emotional intelligence and real-time responsiveness.

Summary:

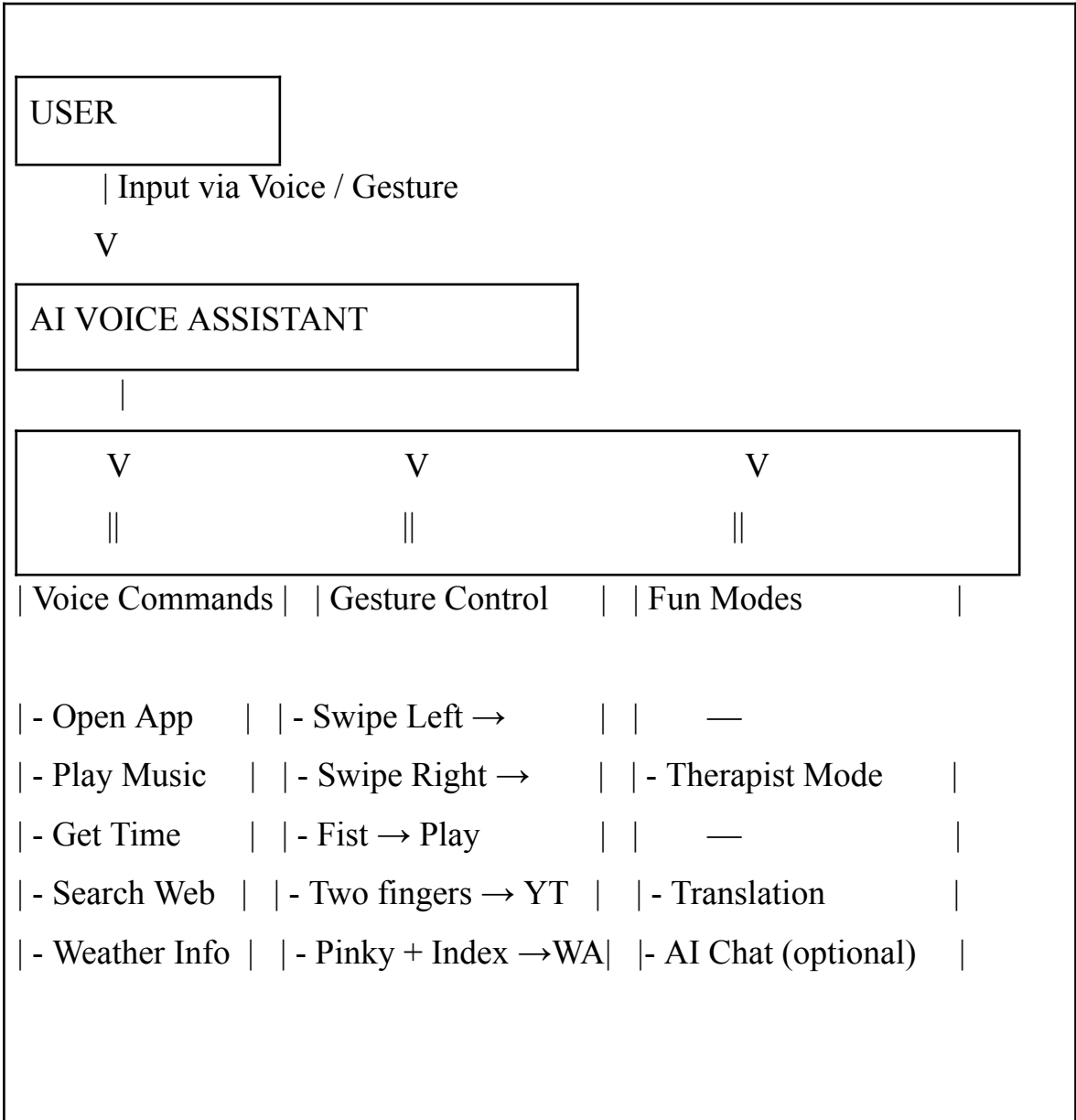
<u>Module</u>	<u>Technology Used</u>	<u>Purpose</u>
Voice Input	speech_recognition	Recognize user speech
TTS Output	pyttsx3	Speak out responses
Gesture Recognition	MediaPipe, OpenCV	Track hand gestures
Mode Switching	State flags	Change assistant behavior
Translator	googletrans	Multilingual support
App Control	pyautogui, subprocess	Automate PC apps

UML Design Overview

We'll explain the design using three uml diagrams:

1. Use Case Diagram:

This shows the interaction between the **User** and the **Assistant System**.



2. Class Diagram (Table Representation):

<u>Class / Module</u>	<u>Attributes</u>	<u>Methods / Functions</u>
VoiceAssistant	mode, wake_word, roast_mode	recognize_speech(), speak(), process_command()
ModeController	therapist_mode	toggle_roast_mode(),get_therapy()

GestureController	—	detect_gesture(), control_music()
AppController	—	open_app(name), close_app(name)
TranslatorModule	—	translate_text(command)

3. Activity Diagram (Step-by-Step in Table):

<u>Step</u>	<u>Description</u>
Start Assistant	The assistant starts listening for input.
Wait for Wake Word	Waits for "Hey Mike" / "Hello Jack".
Capture Command	Converts speech to text.
Process Command	Matches the input with predefined command sets.
Execute Module	Triggers music control, translation, jokes, gestures, app open, etc.
Speak Back to User	Responds via voice using <code>speak()</code> function.
Loop Back	Returns to listening state for next command.

Methodology and Algorithm:

Overview of Methodology:

The proposed voice assistant operates as a multi-modal intelligent system capable of accepting commands through voice and gesture, while providing context-aware responses using natural language. It integrates multiple subsystems like speech recognition, gesture detection, task automation, and interactive AI-based personality modules such as Roast Mode, Therapist Mode, and Sarcasm Mode

The methodology is broken into the following stages:

The JARVIS assistant functions through a combination of voice command processing, gesture control, and intelligent mode-based behavior. Voice command processing begins by capturing audio input from the user through the microphone using Python's `speech_recognition` library. This audio is then converted into text via Google Speech-to-Text API, where the assistant scans for specific keywords or phrases to determine the user's intent—such as playing music, telling a joke, or opening an application—and routes the command to the appropriate function. Simultaneously, the system features a gesture control module that uses a real-time webcam feed, employing MediaPipe Hands for hand landmark detection and OpenCV for gesture recognition. It analyzes wrist and fingertip positions to identify predefined gestures like a swipe left for the next song or a two-finger pose to launch YouTube. This visual input allows for hands-free interaction and accessibility. Additionally, the assistant offers various operational modes: Voice Mode, which uses only speech input; Gesture Mode, which responds to hand gestures via the camera; and Therapist Mode, which engages users with motivational or wellness-focused quotes. These modes provide a highly personalized, engaging, and flexible user experience.

Task Execution Layer

Based on the user command, corresponding tasks are triggered:

- **System Control:** Volume, brightness, shutdown, etc.
- **App Control:** Open/close Notepad, Chrome, etc.
- **Entertainment:** Play music, tell a joke, open YouTube.

- **Communication:** Send WhatsApp messages, voice calls.
- **Modes:** therapy, sarcasm, AI chat (optional).

Response Generation

After the action, the assistant uses text-to-speech (pyttsx3 or gTTS) to:

- Confirm actions.
- Share results (e.g., translation output, time, jokes).
- Deliver responses based on personality mode.

Algorithms Used:

<u>Feature</u>	<u>Algorithm / Approach</u>
Speech Recognition	Google Speech API (NLP + probabilistic matching)
Command Detection	String matching, Regex for key phrases
Gesture Detection	Euclidean Distance & Angle Detection with MediaPipe landmarks
Wake Word Detection	Keyword spotting using simple substring check
Text-to-Speech (TTS)	pyttsx3 for offline, gTTS for multilingual responses
Translation	Google Translator API (googletrans)

Therapy Modes	Random selection from a response pool (AI personality emulation)
Multimodal Switch	Boolean flags (e.g., <code>mode = "gesture"</code>)
Reverse Command Prediction	Simple reversal and lookup of command structure

Tools & Libraries Used:

<u>Library</u>	<u>Purpose</u>
speech_recognition	Speech input
pyttsx3 / gTTS	Voice output
mediapipe	Gesture recognition
cv2 (OpenCV)	Camera control and image processing
googletrans	Language translation
webbrowser, os	Task automation
pyautogui	GUI automation (mouse, keyboard control)
datetime	Time-based responses

random	Randomly select jokes/roasts/therapy
lines	

Implementation

The system implementation phase involves translating the designed features and algorithms into a fully working and interactive voice-based assistant, capable of understanding and executing user commands using voice and gesture modalities.

1. Voice Input & Command Recognition:

Speech-to-Text:

- Utilized the `speech_recognition` library.
- The assistant listens continuously for the wake word (e.g., “Hey Mike” or “Hello Jack”).
- Once activated, it processes the spoken sentence and converts it into text.

Command Matching:

- Predefined command sets are compared using string matching.
- Used conditional statements (if-elif) to determine the action.

2. Gesture Mode:

Libraries Used: mediapipe, opencv-python

Implementation Flow:

- The camera feed is accessed using OpenCV.
- Hand landmarks are detected using MediaPipe’s **Hands** module.

- Specific gestures are mapped to actions:
- Fist → Play/Pause music
- Swipe Left → Next song
- Swipe Right → Previous song
- Two-finger gesture → Open YouTube
- Pinky + index finger → Open WhatsApp
- Three-finger (switch) → Switch back to Voice Mode

3. Voice Response Output:

- **Offline Voice:** Implemented using pyttsx3 for system-generated TTS responses.
- **Multilingual Support:** Implemented using gTTS (Google Text-to-Speech) for Indian language translations.
- **Voice Gender Toggle:**
 - Male voice for normal commands or foreign languages.
 - Female voice for Indian languages or emotional interactions.

4. Personality Modes:

- **Therapist Mode:**
 - Provides motivational and calming responses.
 - Random selection from a curated emotional support message list

5. Task Automation:

- **System-Level Functions:**

Screenshots, volume/brightness control, shutdown/restart/sleep are handled using pyautogui and os modules.

- **Application Launching:**

Launch/Close apps like Notepad, Chrome, WhatsApp, File Explorer using subprocess and os.system.

- **Search & File Access:**

File lookup and read-out functionality with os.walk.

- **Browser-Based Actions:**

Used web browser for searching Google or playing songs on YouTube.

6. Language Translation Feature:

- **Input:** “Translate [phrase] to [language]”

- **Tool:** googletrans library

- **Process:**

Translates given text into the target language.

Speaks the translated result using an appropriate voice.

7. Gesture-to-Document Display:

- When gesture mode is activated, a .docx document is automatically opened.

- This file displays all the available gesture commands for quick references.

Results and Analysis

The implementation of the intelligent voice assistant, enhanced with gesture recognition and emotion-driven interaction modes, led to promising results across multiple areas.

1. Feature Testing:

<u>Feature</u>	<u>Expected Behavior</u>	<u>test Result</u>
Voice Recognition (Wake Word)	Detect "hello jack" / "hey mike"	Success
Command Recognition	Understand and execute valid commands	Success
Music Control via Voice	Play/pause/next/previous	Success
Gesture Control (Media)	Hand gestures trigger media actions	Success
App Opening via Voice	Launch apps like Chrome, WhatsApp	Success
Translation (Indian + Foreign)	Translate & speak in multiple languages	Success
Therapist Mode	Offer emotional support responses	Success
Gesture to Voice Switch	Exit gesture mode via gesture	Success
Search on YouTube/Google	Open browser and run a query	Success

2. Gesture Mode Analysis:

Gesture	Action
Closed Fist	Play/Pause Music
Swipe Left	Next Song
Swipe Right	Previous Song
Index + Middle Finger	Open YouTube
Index + Pinky Finger	Open WhatsApp
Three Finger Raise	Switch to Voice Mode

conclusion

This project was all about building a smart assistant—like your own version of JARVIS—that can understand your voice and do cool things like playing music, opening apps, translating languages, and even switching to gesture mode when needed.

We combined fun features like *Therapist Mode*, and *Gesture Controls* to make it interactive, enjoyable, and also helpful for people who may have physical difficulties using a computer.

From capturing screenshots to answering questions and even cracking jokes, the assistant responds just like a friendly digital buddy. We made sure it's not just functional, but also personal and fun to use.

Overall, this system proves how artificial intelligence, when used creatively, can make life easier and more entertaining. And the best part? You can keep improving it with new features and ideas as tech keeps growing!

